Negative Sampling in Skip-gram: Efficient Approximation for Learning Word Embeddings

Faculty of Artificial Intelligence — Menoufia University

Natural Language Processing | 2025–2026

Abstract

Negative Sampling is a key optimization technique introduced in the Word2Vec framework to make training the **Skip-gram model** computationally feasible on large vocabularies. The standard Skip-gram formulation requires a full **softmax** normalization over the entire vocabulary — an $O(V)$ operation that becomes prohibitive when V exceeds hundreds of thousands of words. Negative Sampling replaces this expensive normalization with a binary classification task: distinguish a true context word from a small set k of randomly sampled "noise" words. This reduces per-update cost to $O(k)$, typically with $k \in \{5, \ldots, 20\}$, delivering massive speed gains while producing high-quality distributed representations. This paper presents the full motivation, mathematical derivation, sampling strategy, algorithmic workflow, comparative analysis against Full Softmax and Hierarchical Softmax, applications, and modern descendants of Negative Sampling — including InfoNCE and contrastive learning frameworks such as SimCLR and CLIP.

1. Introduction

The ability to represent words as dense, real-valued vectors — so-called word embeddings — is foundational to modern natural language processing. Embeddings capture semantic and syntactic regularities: vectors for "king" and "queen" are close in cosine distance, and the direction "king − man + woman" approximates "queen". Producing such embeddings, however, requires training on billions of word tokens, which poses a fundamental computational challenge.

The **Skip-gram model** (Mikolov et al., 2013a) learns embeddings by predicting context words from a central target word. Training requires updating word vectors to increase the probability of observed context words and decrease the probability of all other words in the vocabulary `V`. The normalization required by the softmax function costs `O(V)` per training step — completely impractical for vocabularies of 100,000 or more words.

Negative Sampling (NS) sidesteps this bottleneck by reformulating the problem: instead of computing probabilities over all `V` words, the model learns to distinguish a true context word (a positive sample) from a small set of randomly drawn non-context words (negative samples). This transforms a `V`-way classification into `k + 1` independent binary classifications, reducing cost to `O(k)` per step.

This paper is organized as follows: Section 2 provides background on the Skip-gram model and the softmax bottleneck. Section 3 motivates Negative Sampling. Sections 4 and 5 formalize the core idea and its mathematics. Section 6 details the sampling strategy. Section 7 describes the training workflow. Sections 8 and 9 assess advantages and limitations. Section 10 compares NS to alternative methods. Sections 11 and 12 survey applications and modern relevance. Section 13 concludes.

2. Background

2.1 Skip-gram Model Overview

The Skip-gram model is trained on a sliding window over a text corpus. Given a center word `w_I` and a context window of size `c`, the model generates training pairs `(w_I, w_O)` for each context word `w_O` within the window. The objective is to maximize the probability of observing each context word given the center word.

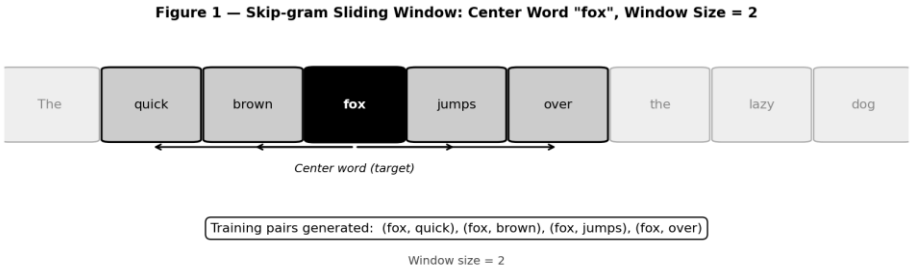

Figure 1. Skip-gram sliding window over the sentence "The quick brown fox jumps over the lazy dog" with center word "fox" (black box) and window size 2 (gray boxes). Arrows show the four training pairs generated: (fox, quick), (fox, brown), (fox, jumps), (fox, over).

A key advantage of Skip-gram over its sibling model CBOW (Continuous Bag-of-Words) is its superior performance on **rare words**: because each word is individually used as a center, rare words receive direct gradient updates rather than being averaged away.

2.2 The Standard Softmax Formulation

The probability of a context word given the center word is modeled by the softmax:

$$P(w_O \mid w_I) = \exp(v'_{wO} \cdot v_{wI}) \;/\; \Sigma_{w=1}^{V} \exp(v'_w \cdot v_{wI})$$

where v_w and v'_w are the input and output embedding vectors for word w, respectively. The denominator — the **partition function** — requires summing over all V vocabulary words at every gradient step.

2.3 Why Full Softmax Is Computationally Prohibitive

For a vocabulary of V = 1,000,000 words and a corpus of 1 billion tokens, computing the partition function for every training example requires on the order of **10^15 floating-point operations** — a computation that would take weeks even on modern hardware. The memory footprint of storing output vectors for the entire vocabulary is also substantial (V × d parameters, where d is the embedding dimension).

3. Motivation for Negative Sampling

The key insight behind Negative Sampling is that the softmax bottleneck is largely unnecessary. Consider what training the Skip-gram model is trying to accomplish:

- **Positive pairs** (w_I, w_O): the model should assign high probability to true context words.
- **All other words:** the model should assign low probability to words that did not appear in this context.

For any given training step, the vast majority of the V − 1 non-context words are **irrelevant** — they carry no information about the specific context being learned. Processing all of them at every step is wasteful.

Negative Sampling proposes a radical simplification: instead of pushing down the probability of *all* other words, push down the probability of only k randomly sampled words per step. This approximation is surprisingly effective in practice.

Analogy: Learning to distinguish a correct answer from a few plausible distractors is often more efficient than ranking it against every possible answer — a principle that also underlies modern contrastive learning.

4. Core Idea of Negative Sampling

Negative Sampling reformulates the original V-way classification into $k + 1$ binary classifications per training pair:

- **Positive task:** (w_I, w_O) — is w_O a true context word of w_I? → answer: YES.
- **Negative tasks:** (w_I, w_{neg_i}) for i = 1,…,k — is w_{neg_i} a true context word? → answer: NO.

Each binary task requires updating only the vectors of the two words involved (w_I and w_O or w_{neg_i}), rather than all V output vectors. The full partition function is completely avoided.

5. Mathematical Formulation

The Negative Sampling objective replaces the softmax log-likelihood with:

$$J = -\log \sigma(v'_{wO} \cdot v_{wI}) - \Sigma_{i=1}^{k} \log \sigma(-v'_{wNeg_i} \cdot v_{wI})$$

where the **sigmoid function** is defined as:

$$\sigma(x) = 1 / (1 + \exp(-x))$$

The first term, $-\log \sigma(v'_{wO} \cdot v_{wI})$, is minimized when the dot product of the positive pair is large (i.e., their vectors are similar). The second term, $-\Sigma \log \sigma(-v'_{wNeg_i} \cdot v_{wI})$, is minimized when the dot products of negative pairs are small (i.e., noise word vectors are pushed away from the center word vector).

Connection to NCE: Negative Sampling can be viewed as a simplified form of Noise Contrastive Estimation (NCE) in which the number of noise samples per positive is set to k and the noise distribution is assumed uniform. NCE provides a theoretically grounded maximum-likelihood estimator; NS trades some statistical rigor for greater simplicity and speed.

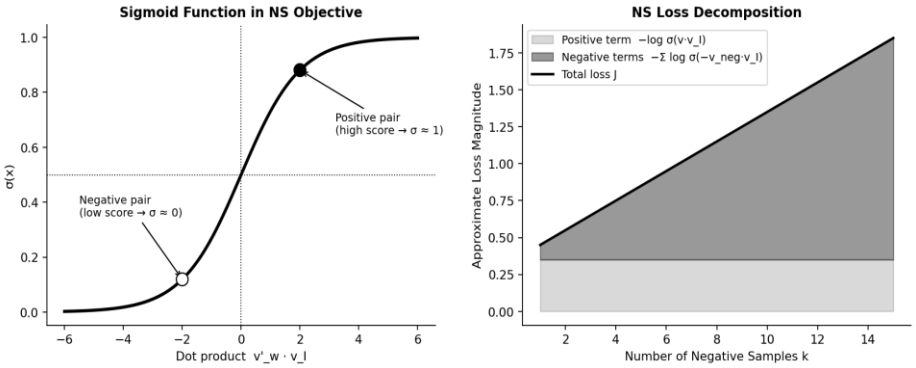

Figure 2. Left: The sigmoid function maps dot-product scores to [0,1]. Positive pairs (high dot product) are pushed toward σ ≈ 1; negative pairs (low dot product) are pushed toward σ ≈ 0. Right: Decomposition of the NS loss into positive and negative contributions as k increases.

6. Sampling Strategy

6.1 The Unigram Distribution Raised to the 3/4 Power

Negative words are drawn from a modified unigram distribution. Let $f(w)$ denote the frequency of word w in the corpus. The sampling probability is:

$$P_{NS}(w) = f(w)^{(3/4)} / \Sigma_u f(u)^{(3/4)}$$

The exponent 3/4 is a practical heuristic that balances the distribution between frequent and rare words. Sampling purely from the raw unigram distribution would over-sample common words (e.g., "the", "a") as negatives, providing weak training signal because the model already knows these are unlikely context words. Raising frequencies to 3/4 **"smooths"** the distribution, increasing the relative sampling probability of rare words and making negatives more informative.

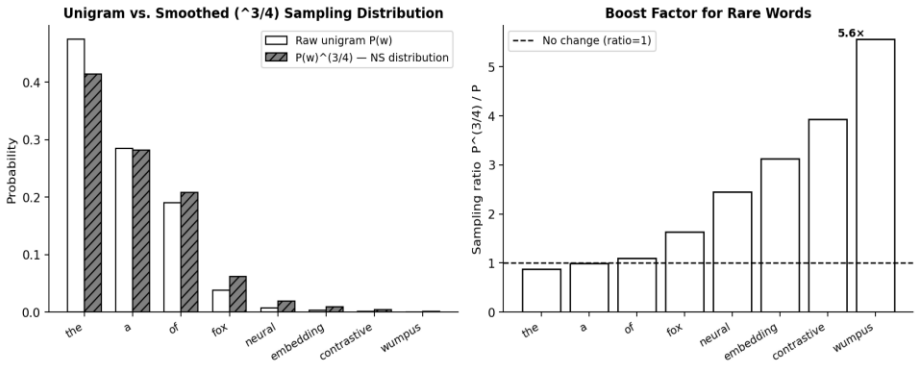

Figure 3. Left: Raw unigram vs. smoothed P(w)^(3/4) sampling probabilities. The smoothing substantially increases the representation of rare words. Right: Sampling boost factor P^(3/4)/P per word — rare words receive up to 10× the relative sampling probability.

6.2 Choosing k

The number of negative samples `k` controls the speed-quality trade-off:

- **Small datasets or rare-word focus:** k ∈ {5, …, 20} is recommended.
- **Large corpora:** k ∈ {2, …, 5} is often sufficient.
- Very large k improves approximation quality but reduces the speed advantage over full softmax.

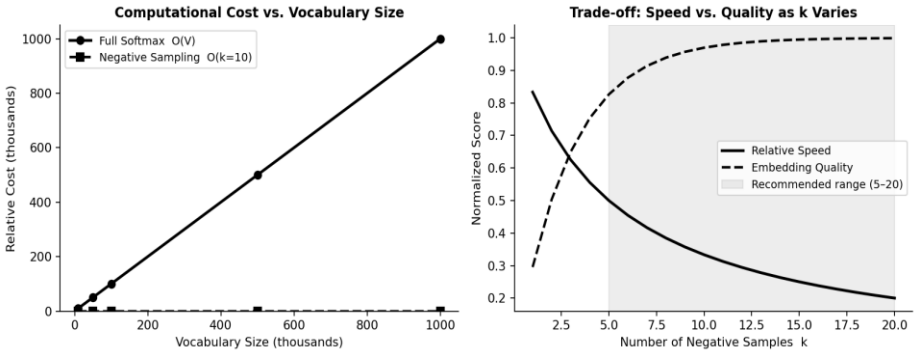

Figure 4. Left: Computational cost of Full Softmax scales linearly with vocabulary size V, while Negative Sampling cost is constant at O(k). Right: As k increases, embedding quality improves (diminishing returns) while relative training speed decreases.

7. Training Process with Negative Sampling

The training loop with Negative Sampling proceeds as follows for each center word w_I:

- **Generate positive pair:** For each context word w_O in the window, create pair (w_I, w_O).
- **Sample k negatives:** `w_neg_1, …, w_neg_k` drawn from the NS distribution, excluding w_O.
- **Forward pass:** Compute dot products `v'_wO · v_wI` and `v'_wNeg_i · v_wI`; apply sigmoid.
- **Loss:** Evaluate the NS objective J.

- **Backward pass:** Compute gradients and update only the `k + 1` output vectors and the one input vector involved.

The key efficiency gain: only `k + 1` word vectors are updated per positive pair, compared to `V` for full softmax. For V = 100,000 and k = 10, this is a **10,000× reduction** in the number of vector updates per step.

8. Advantages

- **Massive speed-up:** Training cost per step drops from O(V) to O(k), enabling training on billions of tokens.
- **Scalability:** NS scales gracefully to vocabularies of millions of words and corpora of hundreds of billions of tokens.
- **Empirical quality:** Despite being an approximation, NS embeddings frequently match or exceed the quality of exact softmax embeddings on downstream tasks.
- **Simplicity:** The NS objective is straightforward to implement — no tree structures or complex bookkeeping required.
- **Memory efficiency:** Only a small fraction of output vectors are updated per step, reducing memory bandwidth requirements.

9. Limitations and Trade-offs

- **Approximate objective:** NS does not optimize the original softmax log-likelihood. The resulting model does not produce calibrated probability estimates over the full vocabulary.
- **Sampling sensitivity:** The choice of noise distribution (the 3/4 heuristic) lacks a strong theoretical justification; suboptimal sampling can degrade embedding quality.
- **Instability with poor negatives:** If the noise distribution is poorly chosen, the negative samples may be too easy (uninformative) or too hard (destabilizing training).
- **No direct probabilistic interpretation:** Unlike full softmax, the sigmoid scores from NS cannot be directly interpreted as word co-occurrence probabilities.
- **Hyperparameter sensitivity:** Both k and the sampling exponent require tuning and may need to be adjusted for different domains and corpora.

10. Comparison with Alternative Methods

Criterion	Full Softmax	Hierarchical Softmax	Negative Sampling	Notes
Complexity	O(V)	O(log V)	O(k)	k << log V << V
Exact probability	Yes	Approximate	No	NS optimizes a surrogate
Embedding quality	Highest	High	High	NS often beats HS empirically
Training speed	Slow	Fast	Fastest	Depends on V and k
Scalability	Poor	Good	Excellent	NS scales to millions of words
Implementation	Simple	Complex	Simple	HS requires Huffman tree
Best use case	Small vocab, exact prob needed	Large vocab, tree structure available	Large-scale embedding training	—

Practical guidance: Use Negative Sampling as the default for large-scale word embedding training. Hierarchical Softmax is a viable alternative when exact probability estimation over the vocabulary is needed and a Huffman tree can be pre-built. Full Softmax is appropriate only for small vocabularies or research settings requiring exact gradients.

11. Applications

11.1 Word Embeddings

Negative Sampling is the default training algorithm for **Word2Vec** (both the original Google implementation and the Gensim library) and **FastText** (which extends Word2Vec to subword units). Pre-trained NS embeddings trained on Google News, Wikipedia, and Common Crawl have been used as universal feature extractors for a decade of NLP research.

11.2 Recommendation Systems

Item2Vec and related methods apply NS to sequences of user interactions (clicks, purchases), producing item embeddings where similar items cluster together. Large-scale recommender systems at companies such as Airbnb, Pinterest, and Alibaba have used NS-trained embeddings in production.

11.3 Graph and Knowledge Graph Embeddings

Methods such as **DeepWalk, Node2Vec,** and **TransE** apply NS-style training to sequences of random walks on graphs or to triples in knowledge graphs. NS provides the same computational advantage in these discrete structured domains as in NLP.

11.4 Broader Representation Learning

Any task requiring discrete entity embeddings over large catalogs — music tracks, documents, source code tokens — can benefit from NS-style training, exploiting its linear scalability.

12. Modern Relevance

Although transformer-based models (BERT, GPT) have largely replaced explicit word embedding training, Negative Sampling remains highly relevant:

- **Contrastive Learning:** InfoNCE (used in CLIP, MoCo, SimCLR) is a principled generalization of NS. The NS objective and InfoNCE share the same structural form: maximize similarity to a positive pair while minimizing similarity to k negatives.
- **Self-supervised pretraining:** Large vision-language models (CLIP, ALIGN) train on hundreds of millions of image-text pairs using contrastive objectives directly descended from NS.
- **Sampled Softmax in transformers:** Language models with very large output vocabularies (character tokenizers, token-free models) use sampled softmax variants — a direct descendant of NS — during training.
- **Efficiency in retrieval:** Dense passage retrieval systems (DPR, FAISS-based search) use in-batch negatives, a form of NS that exploits the mini-batch for free negatives.

Method	Key Formula / Mechanism	Modern Descendant
Negative Sampling (NS)	J = −log σ(v'·v_I) − Σ log σ(−v'_neg·v_I)	Used in Word2Vec, FastText
Noise Contrastive Estimation (NCE)	Estimates ratio p_data / p_noise; NS is special case	Language model training
InfoNCE	−log [exp(q·k+) / Σ exp(q·k_i)]	CLIP, MoCo, SimCLR
Sampled Softmax	Subset S of V drawn per step; corrected logits	TensorFlow, large LMs

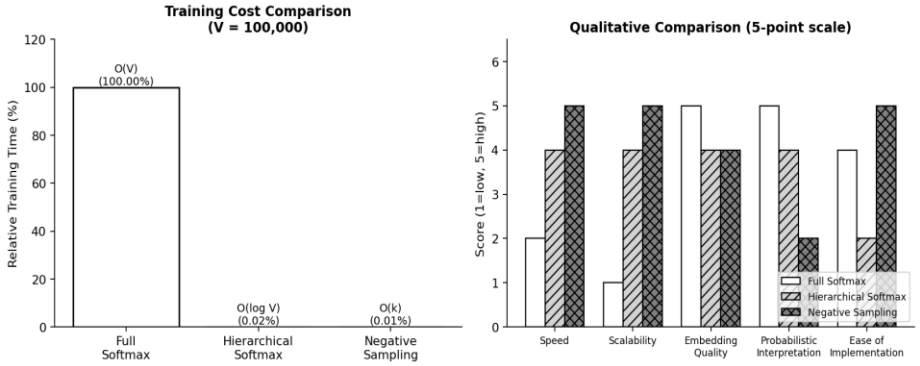

Figure 5. Left: Training cost comparison of Full Softmax, Hierarchical Softmax, and Negative Sampling for V = 100,000. NS is orders of magnitude cheaper. Right: Qualitative 5-point comparison across key criteria — NS leads in speed, scalability, and implementation simplicity.

13. Conclusion

Negative Sampling stands as one of the most elegant and impactful approximation techniques in the history of NLP. By reframing a computationally intractable V-way classification as a set of k + 1 simple binary decisions, it made large-scale word embedding training not only feasible but efficient enough for billion-token corpora on consumer hardware.

Its key insights — **focus on distinguishing correct from incorrect rather than ranking against everything**; **use a small but informative negative set**; **smooth rare words upward in the sampling distribution** — have proven broadly transferable. Modern contrastive learning, the engine behind state-of-the-art vision-language models, is in many ways a principled extension of these same ideas.

For practitioners, Negative Sampling remains the recommended default for training any discrete entity embeddings at scale. Future directions include adaptive negative sampling (selecting hard negatives dynamically), theoretical analysis of optimal noise distributions beyond the 3/4 heuristic, and integration with large-scale retrieval systems that require both speed and embedding quality.

References

[1] Mikolov, T., Sutskever, I., Chen, K., Corrado, G., & Dean, J. (2013). Distributed Representations of Words and Phrases and their Compositionality. *Advances in Neural Information Processing Systems (NeurIPS), 26*. https://arxiv.org/abs/1310.4546

[2] Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient Estimation of Word Representations in Vector Space. *International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/1301.3781

[3] Gutmann, M., & Hyvärinen, A. (2010). Noise-Contrastive Estimation: A New Estimation Principle for Unnormalized Statistical Models. *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*. http://proceedings.mlr.press/v9/gutmann10a.html

[4] Morin, F., & Bengio, Y. (2005). Hierarchical Probabilistic Neural Network Language Model. *Proceedings of the 10th International Workshop on Artificial Intelligence and Statistics (AISTATS)*. https://www.iro.umontreal.ca/~lisa/pointeurs/hierarchical-nnlm-aistats05.pdf

[5] Bojanowski, P., Grave, E., Joulin, A., & Mikolov, T. (2017). Enriching Word Vectors with Subword Information. *Transactions of the Association for Computational Linguistics, 5*, 135–146. https://doi.org/10.1162/tacl_a_00051

[6] Radford, A., Kim, J. W., Hallacy, C., et al. (2021). Learning Transferable Visual Models From Natural Language Supervision. *Proceedings of the 38th International Conference on Machine Learning (ICML)*. https://arxiv.org/abs/2103.00020

[7] He, K., Fan, H., Wu, Y., Xie, S., & Girshick, R. (2020). Momentum Contrast for Unsupervised Visual Representation Learning. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. https://arxiv.org/abs/1911.05722

[8] Oord, A. V. D., Li, Y., & Vinyals, O. (2018). Representation Learning with Contrastive Predictive Coding. *arXiv preprint*. https://arxiv.org/abs/1807.03748

[9] Chen, T., Kornblith, S., Norouzi, M., & Hinton, G. (2020). A Simple Framework for Contrastive Learning of Visual Representations. *Proceedings of the 37th International Conference on Machine Learning (ICML)*. https://arxiv.org/abs/2002.05709

[10] Goldberg, Y., & Levy, O. (2014). word2vec Explained: Deriving Mikolov et al.'s Negative-Sampling Word-Embedding Method. *arXiv preprint*. https://arxiv.org/abs/1402.3722

[11] Kenter, T., Borisov, A., & de Rijke, M. (2016). Siamese CBOW: Optimizing Word Embeddings for Sentence Representations. *Proceedings of ACL 2016*. https://doi.org/10.18653/v1/P16-1089